COURSE PROJECT PROPOSAL

Data Science

Multivariate Industrial Process Anomaly Detection: Isolation Forest vs. PCA Reconstruction Error vs. Rule-Based Thresholds, with TreeSHAP Attribution, on the Tennessee Eastman Process

1. Project Overview & Problem Statement

Industrial equipment is monitored by dense arrays of correlated sensors, and real failures rarely announce themselves as a single sensor crossing a single fixed limit — they propagate across connected units, showing up as a joint, correlated shift across several sensors before any one of them individually looks abnormal. This project builds an unsupervised anomaly-detection pipeline for a genuinely multivariate industrial process, trained only on healthy operating data, and explicitly compares three detection philosophies against each other: independent per-sensor rule thresholds, a linear-subspace method (PCA reconstruction error), and a non-linear partitioning method (Isolation Forest).

This is a software-only, data-science project. An earlier draft explored a physical Arduino demonstration as a proof-of-concept; that component has been removed entirely (Section 6) so that project time, and any evaluation discussion, stays focused on the modeling pipeline rather than on hardware wiring or sensor calibration questions unrelated to the data science being assessed.

This dataset is the Tennessee Eastman Process (TEP), a benchmark simulation of an actual industrial chemical plant — a reactor, condenser, vapor/liquid separator, recycle compressor, and product stripper — with 52 real process sensors and documented, propagating fault behavior across those units.

2. Project Objectives

Objective 1 — Multivariate Feature Preparation: ingest and scale a real multi-sensor industrial process dataset, preserving the cross-sensor correlation structure the rest of the project depends on.

Objective 2 — Non-Linear Unsupervised Detection: train an Isolation Forest strictly on fault-free operating data.

Objective 3 — Linear-Subspace Unsupervised Detection: fit a PCA model on the same fault-free data and use reconstruction error as a second, independent anomaly score, testing whether the process's healthy behavior is adequately captured by a linear subspace or genuinely requires non-linear modeling.

Objective 4 — Rule-Based Baseline: implement an independent per-sensor threshold system, tuned as fairly as the two learned models, as the explicit answer to “why not just use if-else?”

Objective 5 — Root-Cause Attribution: apply TreeSHAP to the Isolation Forest and per-variable reconstruction contribution to the PCA model, so every flagged sample comes with an explanation from each learned model.

Objective 6 — Three-Way Comparative Evaluation: report Precision, Recall, F1-Score, and Early-Warning Lead Time for all three systems, per fault type, producing a complete story: rules vs. linear subspace vs. non-linear partitioning.

3. Dataset Specifications

3.1 Primary Dataset Identity

This project uses the Tennessee Eastman Process (TEP) simulation dataset, specifically the large-sample extension published by Rieth, Amsel, Tran & Cook (2017), "Additional Tennessee Eastman Process Simulation Data for Anomaly Detection Evaluation," hosted on Harvard Dataverse (DOI: 10.7910/DVN/6C3JR1), built on the original process defined by Downs & Vogel (1993).

The dataset ships as four R data files — fault_free_training, fault_free_testing, faulty_training, and faulty_testing — each with 55 columns: faultNumber (0 for fault-free, 1–20 per fault), simulationRun (a distinct random seed per run), sample (the time index, sampled every 3 minutes), and 52 process variables (41 continuous measurements plus 11 manipulated variables).

3.2 Why TEP Instead of a Single-Asset Dataset

- Genuine multi-unit industrial process, not a single machine: TEP's five interconnected units mirror the interconnected stages of a real industrial plant.
- Higher-dimensional, well-documented correlation structure: 52 variables give real room for joint, multi-sensor anomalies that a per-sensor rule system structurally cannot see.
- Established literature precedent for this exact three-way comparison style (rule-based, PCA, and tree-based methods on TEP), so results can be sanity-checked against existing published work.

3.3 Alternate Datasets Considered

Two other genuinely industrial, multivariate/physical datasets were evaluated. Both are viable as optional secondary validation if time allows, but neither replaces TEP as the primary dataset:

Dataset	Sensor Profile	Fit for This Project
Tennessee Eastman Process (TEP) — PRIMARY	52 variables: 41 continuous process measurements + 11 manipulated variables, across a reactor, condenser, vapor/liquid separator, recycle compressor, and product stripper	Genuinely industrial (chemical process plant), strongly correlated multivariate sensors by design, 20 documented fault types with known cascading propagation through connected units.
NASA IMS Bearing Dataset — optional validation/stretch	Vibration (accelerometer) signals from 4 bearings on a common shaft	Real physical rotating-machinery data, but signal-processing-heavy and univariate-per-bearing, shifting the project toward signal processing rather than multivariate tabular anomaly detection.
SKAB (Skoltech Anomaly Benchmark) — optional validation/stretch	~8 sensors on a real water circulation testbed	Real hardware and genuinely multivariate, but its small sensor count gives a narrower story than TEP's 52-variable, multi-unit process.

3.4 Scope Boundary — Variable and Fault Subset

The full Rieth dataset spans 500 simulation runs per condition and is several gigabytes across its four files — too large, and larger than necessary, for this project's question. This project subsets it as follows:

- Variables: all 52 process variables are retained except XMV(5) (compressor recycle valve) and XMV(9) (stripper steam valve), documented in prior TEP literature as taking near-constant values.

- Fault types: IDV(1) and IDV(6) (large, easily separable step-type disturbances — a shared floor all three systems should catch) and IDV(4) and IDV(8) (subtler, more correlated fault signatures where the rule-based baseline is expected to underperform and the PCA-vs-Isolation-Forest comparison becomes informative).
- Simulation runs: approximately 100 of the 500 available runs (70 training / 30 held-out for fault-free; a comparable held-out sample per faulty condition).

3.5 Data Acquisition Workflow — Handling the .RData / pyreadr Risk

The raw Rieth et al. files are multi-gigabyte .RData objects, and parsing them repeatedly with pyreadr risks out-of-memory failures or slow iteration on a typical laptop. This project treats that as a Phase 1, Week 1 task rather than an ongoing risk: each raw file is loaded exactly once, immediately subset to the ~100 selected simulation runs (Section 3.4), and written out as Parquet. All subsequent development, training, and evaluation work reads only from these Parquet files; the raw .RData files are never reloaded once this conversion is complete.

3.6 Data Splitting Strategy

All three detectors — Isolation Forest, PCA, and the rule-based baseline — are fit exclusively on fault-free training runs. The split is performed at the simulationRun level, not by row, since consecutive samples within a run are highly autocorrelated. Held-out fault-free runs verify each system's false-alarm rate; held-out faulty runs (per selected fault type) are used to compute detection performance and lead time, using each run's documented fault-introduction sample as the lead-time reference point.

4. Technical Architecture & Methodology

4.1 Pipeline Overview

Data Acquisition & Immediate Parquet Conversion → Subsetting & Grouped Split → Feature Scaling → [Rule-Based Thresholds] + [PCA Reconstruction Error] + [Isolation Forest] fit in parallel on fault-free data → TreeSHAP / Reconstruction-Contribution Attribution → Three-Way Comparative Evaluation → Results Viewer

4.2 Phase 1 — Rule-Based Thresholds

For every retained variable, a threshold band is fit on fault-free training data (mean ± k standard deviations, k chosen so this system's held-out false-alarm rate matches the other two as closely as possible). A sample is flagged if any single variable falls outside its band — by construction, unable to detect a joint shift where every individual variable stays within its own normal range.

4.3 Phase 2 — PCA Reconstruction Error

PCA is fit on the scaled fault-free training data, retaining the number of principal components that explain approximately 90–95% of variance. For a new sample x, the reconstruction error $\|x - x'\|^2$ (where x' is x projected onto the retained components and back) measures how far the sample lies from the learned linear subspace of normal behavior. A percentile/IQR threshold on this error, fit on healthy training data, produces a binary flag. Per-variable squared reconstruction error gives a natural, cheap attribution signal, directly comparable to Isolation Forest's SHAP ranking.

4.4 Phase 3 — Isolation Forest

Isolation Forest is fit on the same scaled, fault-free training features, with no linearity assumption. Its anomaly score, based on isolation path length, is converted to a binary flag via a percentile/IQR threshold computed from the healthy training-score distribution.

4.5 Phase 4 — Attribution

TreeSHAP is applied to the fitted Isolation Forest for every flagged sample. For PCA, the per-variable squared reconstruction error is reported as a ranked contribution list. Comparing the two rankings on the same flagged samples is itself a small, informative result: agreement suggests a fault with a strong linear signature, and disagreement suggests one the non-linear model is capturing that the linear one is not.

5. The Three-Way Comparison: What Each System Assumes, and Why That Matters

A natural objection to any anomaly-detection project is: couldn't a handful of if-else threshold checks do the same job? For single-variable extreme faults, yes — and this project does not hide from that: IDV(1) and IDV(6) are included specifically because all three systems are expected to catch them, which is itself a useful, honest result showing the rule-based baseline is competently built, not artificially weakened.

The real payload is in how the three systems differ in what they assume about “normal”:

System	What It Assumes About Normal	Expected Strength / Weakness
Rule-Based Thresholds	Each variable independently stays within its own healthy range	Catches large single-variable faults (IDV(1), IDV(6)); structurally blind to joint, correlated shifts (IDV(4), IDV(8))
PCA Reconstruction Error	Healthy data lies near a low-dimensional linear subspace	Catches faults that break the linear correlation structure; may miss faults that are non-linear in nature
Isolation Forest	Healthy data forms a dense region that is hard to isolate by recursive partitioning; no linearity assumption	Captures both linear and non-linear joint anomalies; the comparison's main hypothesis is that this generalizes best across all 4 fault types

IDV(4) and IDV(8) are included specifically to surface where these assumptions diverge in practice. If PCA performs comparably to Isolation Forest on all four fault types, that is a legitimate and interesting finding — it would suggest TEP's fault signatures are largely linear in this feature space. If Isolation Forest clearly outperforms PCA on IDV(4)/IDV(8), that is the evidence for genuine non-linear structure. Either outcome is reported as obtained; the evaluation in Section 7 is run once per system on the held-out set, not adjusted after the fact.

6. Explicit Exclusion: No Hardware Component

An earlier draft of this proposal included a small Arduino-based physical proof-of-concept (potentiometers standing in for sensor channels, driving LEDs based on live model inference) as a demonstration layer. That component has been removed entirely from this proposal. This is a data science project, and a hardware demonstration risks shifting evaluation attention toward wiring, sensor calibration, and embedded-systems

questions that are unrelated to the modeling work being assessed. All demonstration value is instead delivered through the results viewer (Section 8, Week 10), which presents the same core evidence — detection timing, thresholds, and attribution — without introducing hardware as a discussion surface.

7. Evaluation Protocol

All evaluation is performed on held-out simulation runs never used in training, identically for all three systems so they are directly comparable:

- Precision, Recall, and F1-Score per fault type (IDV(1), IDV(4), IDV(6), IDV(8)), plus false-alarm rate on held-out fault-free runs, for each of the three systems.
- Early Warning Lead Time: the gap between each system's first sustained flag (N consecutive samples above threshold) and the run's documented fault-introduction sample, averaged per fault type, per system.
- A single summary table — rows: the 4 fault types; columns: Rule-Based, PCA, Isolation Forest, each with P/R/F1/lead-time — is the central result artifact of this project.

8. Implementation Roadmap & Milestones

A 12-week timeline, sized for one author working part-time alongside other coursework:

Week	Milestone	Deliverable / Exit Criteria
1	Acquire Rieth et al. (2017) TEP data; immediately subsample & convert to Parquet	Raw .RData files loaded via pyreadr exactly once; ~100 selected simulation runs (Section 3.4) written to Parquet; raw .RData files not touched again for the rest of the project
2	Variable & fault subset selection; exploratory correlation analysis	Working subset defined (~50 variables × 4 fault types + fault-free); correlation heatmap documenting cross-sensor relationships
3	Grouped train/holdout split by simulationRun; feature scaling	70/30 fault-free split, unit-tested for no run overlap; StandardScaler fit on training runs only
4	Train Isolation Forest + percentile/IQR threshold	Fitted model; anomaly-score distribution and threshold plotted for held-out fault-free runs
5	Train PCA reconstruction-error model + threshold	PCA fit on training runs (components chosen to explain ~90–95% variance); reconstruction-error distribution and IQR threshold computed on the same held-out fault-free runs
6	Implement rule-based per-variable threshold baseline	Independent threshold band per variable, tuned so its held-out false-alarm rate matches the other two systems as closely as possible
7	Apply all three systems to held-out faulty runs (4 fault types)	Anomaly scores/flags from Isolation Forest, PCA, and the rule-based baseline logged per sample for every held-out faulty run
8	TreeSHAP attribution (Isolation Forest) + per-variable reconstruction contribution (PCA)	Ranked variable-contribution lists for representative flagged samples from both models, enabling a direct explainability comparison

9	Comparative evaluation: P/R/F1 + lead time, three systems × four fault types	Summary comparison table (Section 7) fully populated; IDV(1)/IDV(6) expected convergence and IDV(4)/IDV(8) expected divergence checked against results
10	Lightweight results viewer (precomputed)	Fault/run selector, score-vs-cycle plot with threshold and true fault-introduction point, SHAP/PCA attribution view, metrics readout — all reading precomputed results, no live pipeline
11	Report drafting + plot consolidation	First full report draft; all figures finalized
12	Report finalization, reproducibility check, presentation rehearsal	Final report; code runs end-to-end on a clean checkout producing the same headline metrics; presentation rehearsed around the comparison table and SHAP plots

9. Explicit Scope Boundaries

In Scope (Core, Graded)	Out of Scope
TEP: 4 fault types (IDV(1), IDV(4), IDV(6), IDV(8)) + fault-free baseline	All 20 TEP fault types; fault diagnosis (identifying *which* fault) as opposed to detection
~50 of 52 process variables (2 near-constant manipulated variables dropped)	Full unpruned 52-variable set
~100 of 500 available simulation runs (70 train / 30 holdout)	The full 500-run Rieth dataset
Three-way detector comparison: Isolation Forest, PCA reconstruction error, rule-based thresholds	Any deep-learning detector (LSTM-AE, autoencoder) as a fourth model
TreeSHAP (Isolation Forest) + per-variable reconstruction contribution (PCA) for explainability	Multiple explainability methods per model / cross-library XAI comparison
Static/precomputed results viewer	A live, continuously-updating dashboard
Software-only pipeline, entirely on TEP data	Physical hardware, embedded sensors, or any Arduino/IoT component — cut entirely: this is a data science project and hardware demonstrations invite evaluation questions (wiring, sensor drift, analog filtering) unrelated to the modeling work being graded

10. Tools, Frameworks & Tech Stack

- Language: Python
- Data Ingestion: pyreadr (reads .RData files without requiring an R installation), used once per file per Section 3.5
- Data Manipulation & Metrics: NumPy, Pandas, SciPy
- Modeling: Scikit-Learn (IsolationForest, PCA, StandardScaler)
- Model Interpretability: SHAP (TreeExplainer) for Isolation Forest; per-variable reconstruction error for PCA
- Visualization: Matplotlib, Seaborn

- Environment: Jupyter notebooks for exploration; standalone scripts for the reproducible pipeline

11. Risks & Mitigations

Risk — Dataset size and format: pyreadr parsing gigabyte-scale .RData files can be slow or trigger out-of-memory failures, since pyreadr loads the entire R dataframe into memory before any subsetting is possible. Mitigation: this is addressed structurally, not just noted — Section 3.5 makes immediate Parquet conversion a mandatory Week 1 deliverable, and the raw .RData files are never reloaded afterward. If pyreadr runs out of memory on the full file, fall back to a short R script that loads and subsets the dataframe before export, avoiding the need to hold the full file in Python memory at all:

```
load("fault_free_training.RData")
subset_df <- fault_free_training[fault_free_training$simulationRun <= 100, ]
write.csv(subset_df, "fault_free_training_subset.csv", row.names = FALSE)
```

Run with Rscript extract_subset.R, repeated per file (for the two faulty files, add a filter on faultNumber %in% c(0,1,4,6,8) in the same subsetting line), then load the resulting CSVs into pandas — pyreadr is not needed at all under this path. If R is unavailable, running the single pyreadr load on a machine with more available memory (e.g., a free-tier hosted notebook environment) and downloading only the resulting Parquet file back is an equally acceptable one-time workaround; it does not change the Week 1 milestone, only how that one load is executed.

Risk — Rule-based or PCA baseline built as a strawman: an unfairly weak baseline would inflate Isolation Forest's apparent advantage. Mitigation: both the rule-based thresholds and the PCA reconstruction-error threshold are tuned so their held-out false-alarm rates match as closely as feasible before any fault-detection comparison is run, and this tuning process is documented in the report.

Risk — TreeSHAP compatibility with Isolation Forest's scoring function: SHAP's TreeExplainer is designed around a well-defined tree-ensemble output, and Isolation Forest's path-length-based score is a less standard target; specific shap/scikit-learn version pairings have occasionally triggered mismatches or API warnings with this combination. Mitigation: pin exact package versions at project start (e.g., a fixed scikit-learn and shap release recorded in a requirements file) rather than installing whatever the latest release happens to be that week, and validate compatibility on a tiny toy example before touching real data, in Week 8, before applying it to the full held-out set:

```
from sklearn.ensemble import IsolationForest
import numpy as np, shap

X = np.random.randn(50, 4)
model = IsolationForest().fit(X)
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X[:5])
print(shap_values)  # if this runs without error, proceed to the full dataset
```

If this toy check fails, fall back to permutation importance restricted to flagged anomalies (more compute, no scope change) rather than losing time debugging TreeSHAP internals during the evaluation week.

Risk — PCA component count sensitivity: too few retained components could make PCA fail trivially, overstating Isolation Forest's advantage. Mitigation: choose component count via an explained-variance threshold (90–95%) rather than an arbitrary small number, and report the chosen count and variance explained in the final report.